Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:923001733012**

Aim: To remember the important tasks and forget the less important tasks using LSTM

IDE: Google Colab

Theory:

Countless learning tasks require dealing with sequential data. Image captioning, speech synthesis, and music generation all require that models produce outputs consisting of sequences. In other domains, such as time series prediction, video analysis, and musical information retrieval, a model must learn from inputs that are sequences. These demands often arise simultaneously: tasks such as translating passages of text from one natural language to another, engaging in dialogue, or controlling a robot, demand that models both ingest and output sequentially-structured data.

Recurrent neural networks (RNNs) are deep learning models that capture the dynamics of sequences via *recurrent* connections, which can be thought of as cycles in the network of nodes. This might seem counterintuitive at first. After all, it is the feedforward nature of neural networks that makes the order of computation unambiguous. However, recurrent edges are defined in a precise way that ensures that no such ambiguity can arise. Recurrent neural networks are *unrolled* across time steps (or sequence steps), with the *same* underlying parameters applied at each step. While the standard connections are applied *synchronously* to propagate each layer's activations to the subsequent layer *at the same time step*, the recurrent connections are *dynamic*, passing information across adjacent time steps. As the unfolded view reveals, RNNs can be thought of as feedforward neural networks where each layer's parameters (both conventional and recurrent) are shared across time steps. Like neural networks more broadly, RNNs have a long discipline-spanning history, originating as models of the brain popularized by cognitive scientists and subsequently adopted as practical modeling tools employed by the machine learning community.

The name of LSTM refers to the analogy that a standard RNN has both "long-term memory" and "short-term memory". The connection weights and biases in the network change once per episode of training, analogous to how physiological changes in synaptic strengths store long-term memories; the activation patterns in the network change once per time-step, analogous to how the moment-to-moment change in electric firing patterns in the brain store short-term memories. The LSTM architecture aims to provide a short-term memory for RNN that can last thousands of timesteps, thus "long short-term memory".

A common LSTM unit is composed of a cell, an input gate, an output gate and a forget gate. The cell remembers values over arbitrary time intervals and the three gates regulate the flow of information into and out of the cell. LSTM networks are well-suited to classifying, processing and making predictions based on time series data, since there can be lags of unknown duration between important events in a time series. LSTMs were developed to deal with the vanishing gradient problem that can be encountered when training traditional RNNs. Relative

insensitivity to gap length is an advantage of LSTM over RNNs, hidden Markov models and other sequence learning methods in numerous applications.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Apply LSTM Model
5. Apply the training over the dataset to minimize the loss
6. Observe the cost function vs iterations learning curve

Program (Code):

```
!pip install tensorflow

import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, Dense

# Generate sine wave data

data = np.sin(np.linspace(0, 100, 1000))


# Prepare dataset

def create_dataset(data, time_step=10):

    X, y = [], []

    for i in range(len(data) - time_step):

        X.append(data[i:i+time_step])

        y.append(data[i+time_step])

    return np.array(X), np.array(y)
```

```
time_step = 20

X, y = create_dataset(data, time_step)


# Reshape for LSTM [samples, time steps, features]

X = X.reshape(X.shape[0], X.shape[1], 1)

model = Sequential([

    LSTM(50, activation='relu', input_shape=(time_step, 1)),

    Dense(1)

])

model.compile(optimizer='adam', loss='mse')

model.summary()

history = model.fit(X, y, epochs=10, batch_size=32)

predictions = model.predict(X)

plt.figure(figsize=(10,5))

plt.plot(y, label='Actual')

plt.plot(predictions, label='Predicted')

plt.legend()

plt.title("LSTM Prediction")

plt.show()
```

Results:

To be attached with

a. Model Summary
b. Training and Validation accuracy v/s epochs
c. Training and Validation loss v/s epochs

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:923001733012**

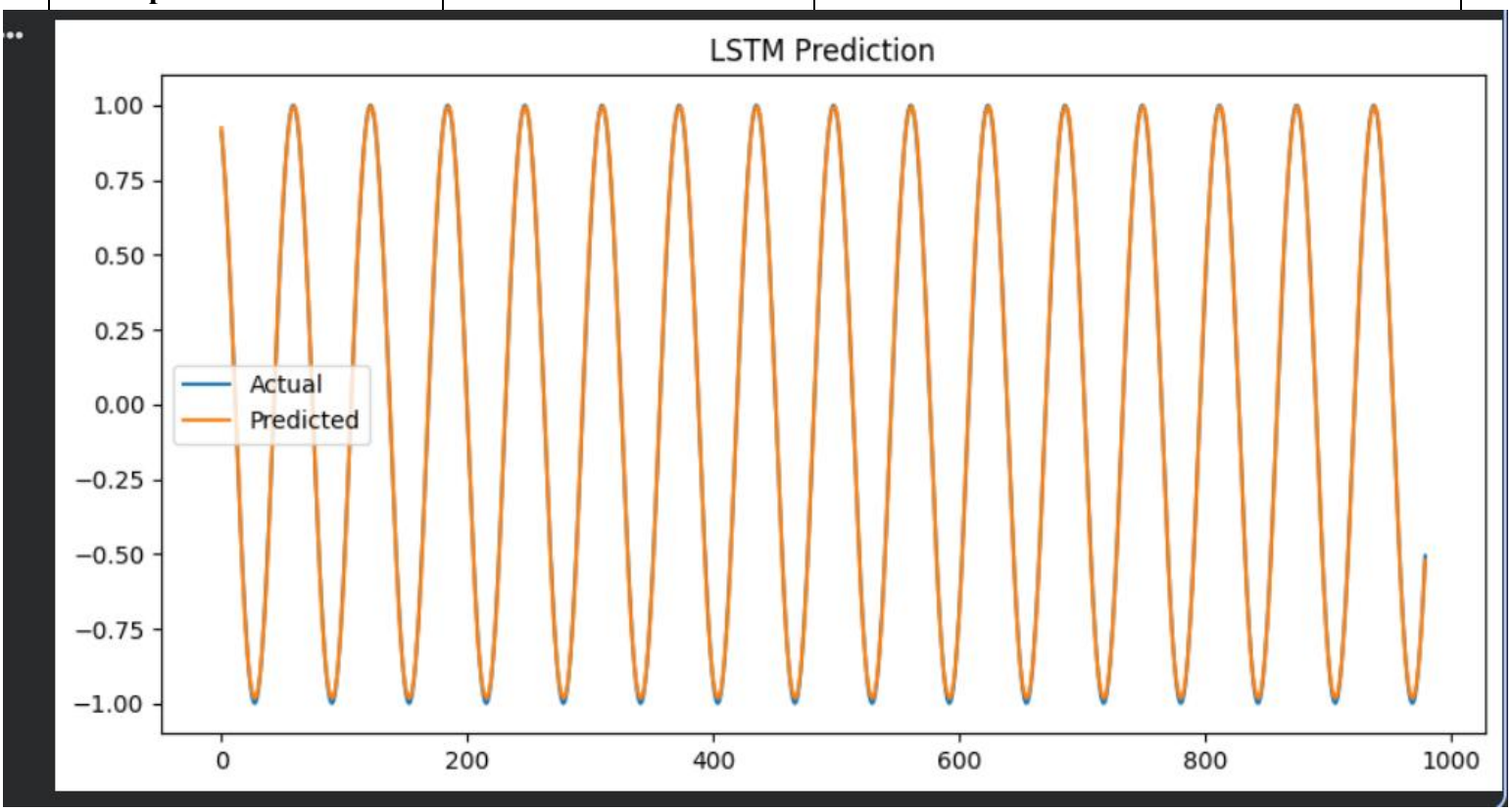
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `ir
  super().__init__(**kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 50)	10,400
dense (Dense)	(None, 1)	51

```
Total params: 10,451 (40.82 KB)
Trainable params: 10,451 (40.82 KB)
Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/10
31/31 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.3121
Epoch 2/10
31/31 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 0.0728
Epoch 3/10
31/31 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 0.0095
Epoch 4/10
31/31 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 0.0029
Epoch 5/10
31/31 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 0.0013
Epoch 6/10
31/31 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 5.2879e-04
Epoch 7/10
31/31 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 4.2791e-04
Epoch 8/10
31/31 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - loss: 2.9100e-04
Epoch 9/10
31/31 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.9992e-04
Epoch 10/10
31/31 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.5127e-04
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:923001733012**



Observation and Result Analysis:

a. Nature of the dataset

The dataset is a synthetic sine wave generated using np.sin. It's a continuous, time-series-like dataset. The create_dataset function transforms this raw sine wave into a supervised learning problem, where X contains sequences of time_step values and y contains the next value in the sequence, which the LSTM model attempts to predict.

b. During Training Process

During the training process, the LSTM model was trained for 10 epochs. In each epoch, the model processed the data in batches of 32 samples. The loss (mean squared error) consistently decreased from 0.3121 in the first epoch to 1.5127e-04 in the final epoch, indicating that the model successfully learned to predict the sine wave pattern.

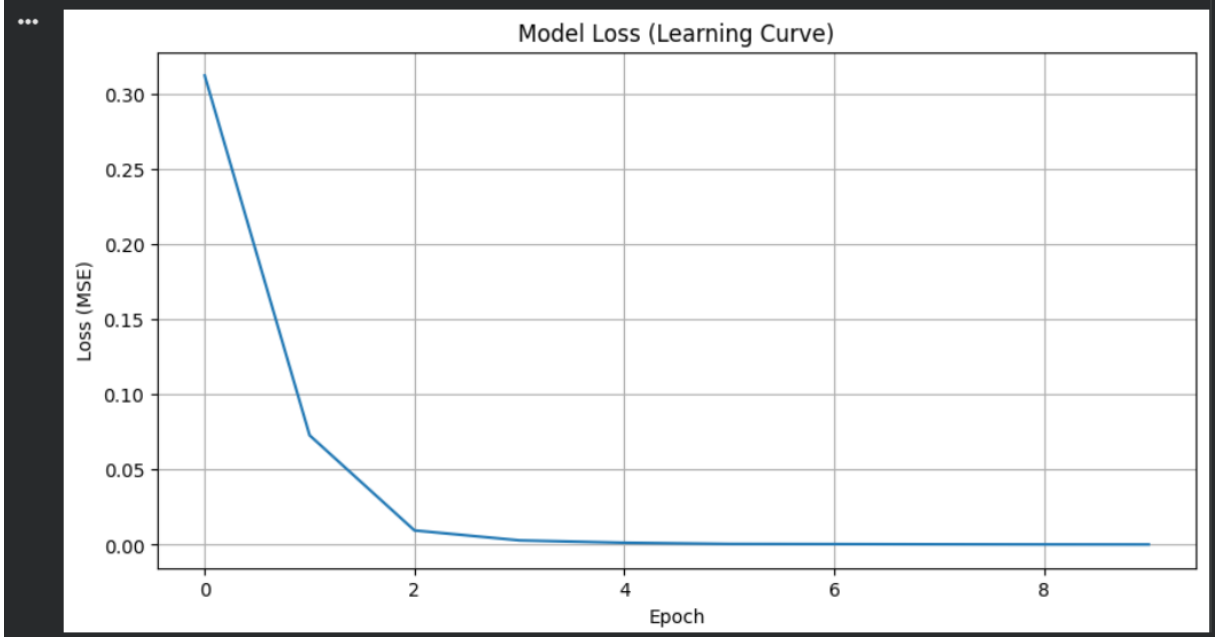
c. After the training Process

After the training process, the model made predictions on the input data. The plot visually compares these predictions against the actual sine wave values. As observed, the predicted curve closely follows the actual sine wave, indicating

that the LSTM model successfully learned the underlying pattern and can accurately predict the next value in the sequence.

d. Observation over the Learning Curve

```
plt.plot(history.history['loss'])
plt.title('Model Loss (Learning Curve)')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE)')
plt.grid(True)
plt.show()
```



The learning curve shows that the model's loss (Mean Squared Error) decreased significantly and rapidly over the training epochs. This indicates that the model was effectively learning and improving its predictions with each epoch, converging to a low error state. The steep initial drop suggests efficient learning, while the flattening curve towards the end implies that the model had largely captured the patterns in the data and further training might yield diminishing returns.

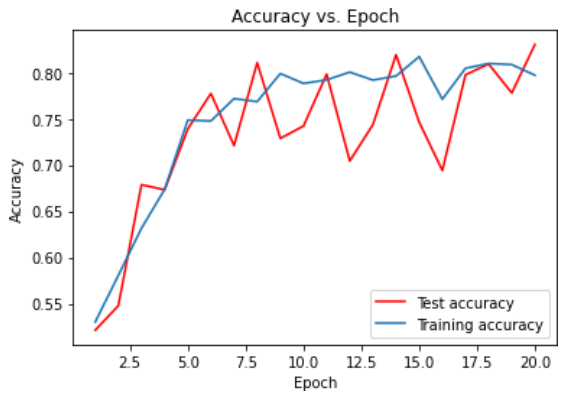
Post Lab Exercise:

a. What is the requirement of LSTM over RNN?

Long Short-Term Memory (LSTM) networks were developed to address some of the fundamental limitations of traditional Recurrent Neural Networks (RNNs), primarily the vanishing and exploding gradient problems, which make it difficult for RNNs to learn long-term dependencies.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:923001733012**

b. Interpretation of graph



What is the meaning of the above graph? How can the graph be smoothened?

- ✓ The graph shows training accuracy (blue) and test accuracy (red) over epochs.
- ✓ Training accuracy increases steadily, which means the model is learning patterns from the training data.
- ✓ Test accuracy fluctuates a lot instead of following a smooth upward trend.

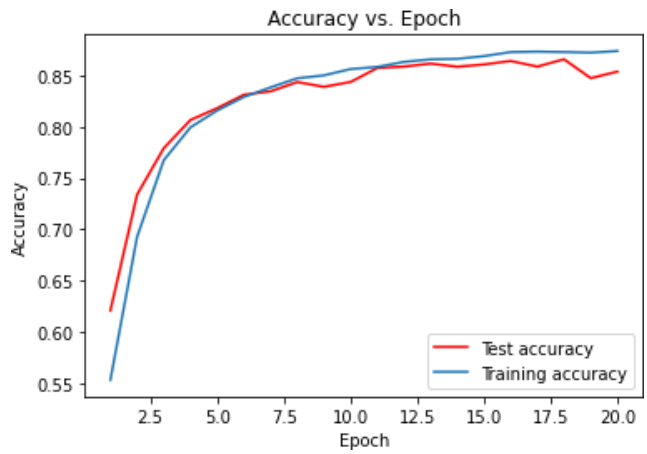
This indicates:

- ✓ The model is learning, but
- ✓ It has high variance / instability on unseen data.

Likely causes:

- ✓ Small dataset
- ✓ Noise in data
- ✓ Learning rate too high

c.



What is the meaning of the above graph? How can the graph be smoothened?

- ✓ Both training and test accuracy increase smoothly and almost overlap.
- ✓ Very small gap between them.
- ✓ This indicates:
  - Good learning
  - Good generalization
  - No significant overfitting or underfitting

Post Lab Activity

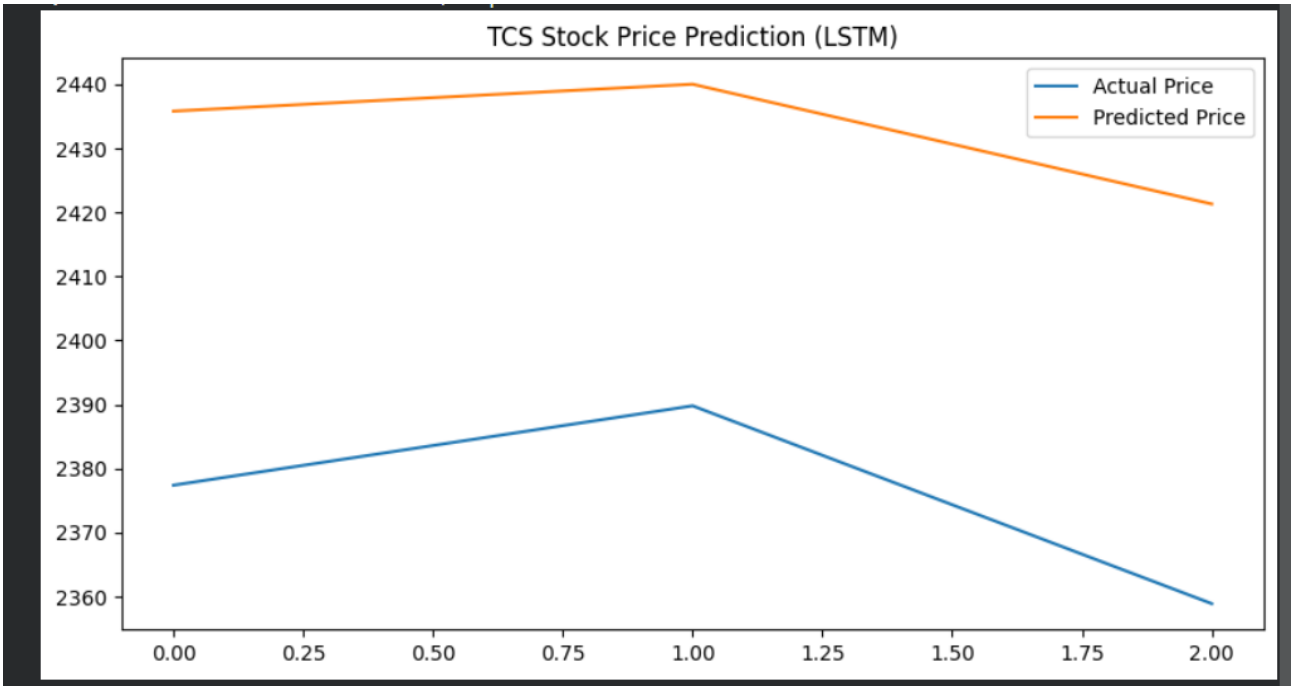
Obtain the prediction value of the TCS stock for the historical data obtained from the link mentioned as: https://in.investing.com/equities/tata-consultancy-services-historical-data

Select the Date Range from (01-01-2020 to 31-03-2024). Select the price range to be as "DAILY" stock values. Train your LSTM model to obtain the predicted value as close as the actual value. Paste the screenshot of the output.

Code:

```
# Install libraries
!pip install pandas numpy matplotlib scikit-learn tensorflow

# Import
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Upload CSV
from google.colab import files
uploaded = files.upload()

# Load dataset
df = pd.read_csv(list(uploaded.keys())[0])

# Keep only Date & Price (Close)
df = df[['Date', 'Price']]
df['Price'] = df['Price'].str.replace(',', '').astype(float)

# Sort data
df = df.sort_values('Date')

# Normalize
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(df[['Price']])

# Create sequences
def create_dataset(data, time_step=60):
    X, y = [], []
    for i in range(len(data)-time_step):
        X.append(data[i:i+time_step])
        y.append(data[i+time_step])
    return np.array(X), np.array(y)

# Reduce time_step as the current dataset is too small
time_step = 5  # Changed from 60 to 5
X, y = create_dataset(data_scaled, time_step)
```

```
# Train-test split
split = int(len(X)*0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Model
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(time_step,1))) # Updated input_shape
model.add(LSTM(50))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mean_squared_error')

# Train
model.fit(X_train, y_train, epochs=10, batch_size=32)

# Predict
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

# Inverse scaling
train_pred = scaler.inverse_transform(train_pred)
y_train = scaler.inverse_transform(y_train)

test_pred = scaler.inverse_transform(test_pred)
y_test = scaler.inverse_transform(y_test)

# Plot
plt.figure(figsize=(10,5))
plt.plot(y_test, label='Actual Price')
plt.plot(test_pred, label='Predicted Price')
plt.legend()
plt.title("TCS Stock Price Prediction (LSTM)")
plt.show()
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To remember the important tasks and forget the less important tasks using LSTM**	
Experiment No: 05	**Date:**	**Enrolment No:923001733012**